

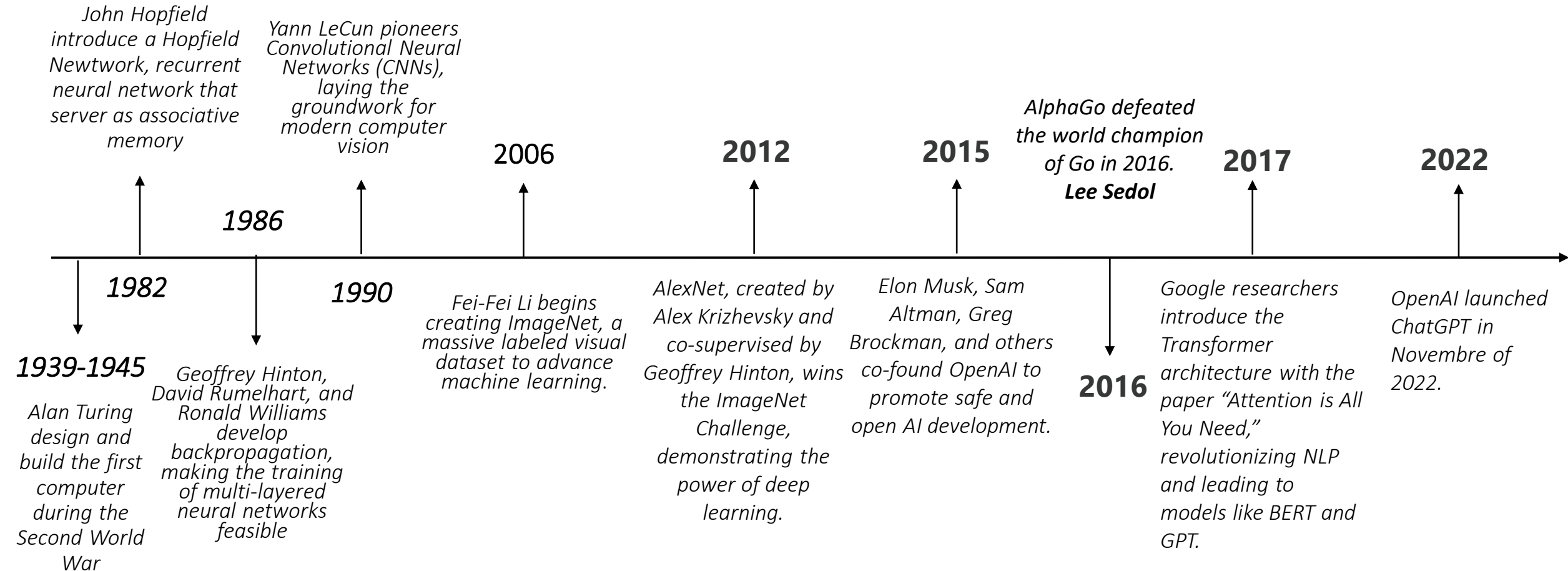
Ingeniería de soluciones con AI

Semestre 2026-1

Docente:
Francisco Macaya

Experiencia de Aprendizaje 1
Fundamentos de AI Generativa y
Prompt Engineering

Historia: Deep Learning



Breakpoint: Attention Is All You Need

Provided proper attribution is provided, Google hereby grants permission to reproduce the tables and figures in this paper solely for use in journalistic or scholarly works.

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com	Noam Shazeer* Google Brain noam@google.com	Niki Parmar* Google Research nikip@google.com	Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com	Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu	Lukasz Kaiser* Google Brain lukaszkaizer@google.com	
Illia Polosukhin* † illia.polosukhin@gmail.com			

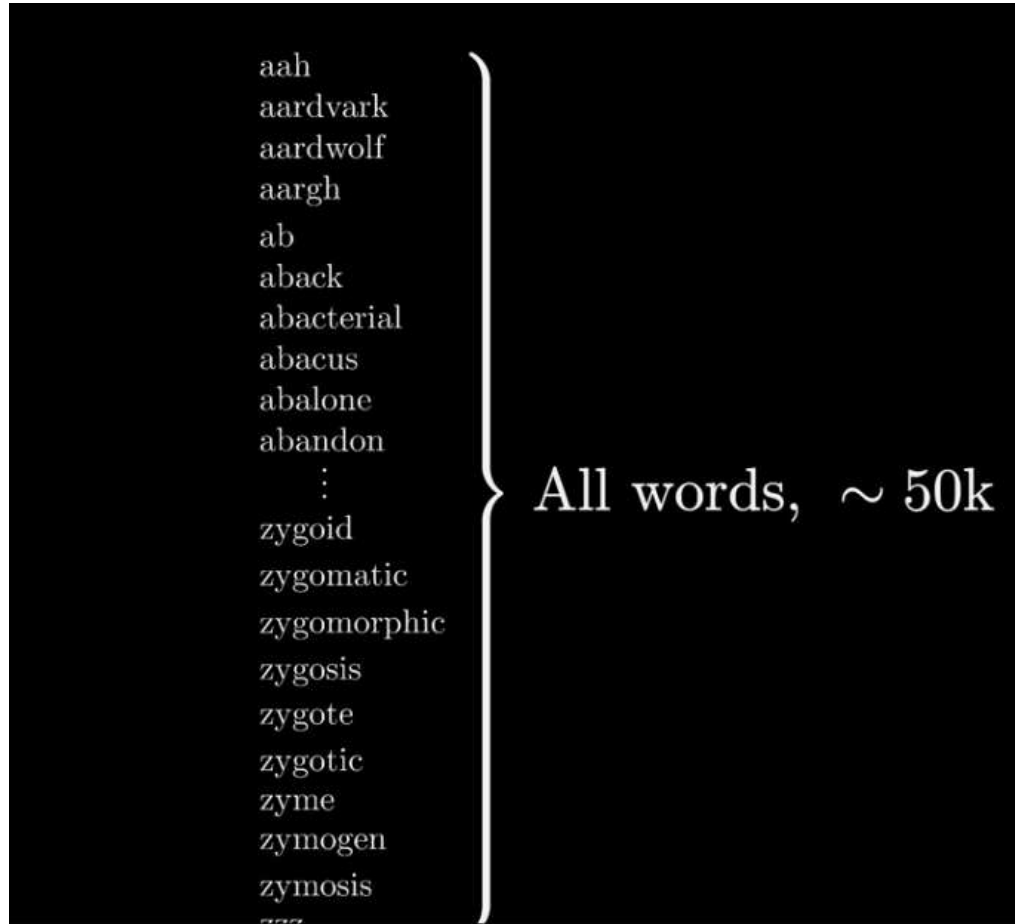
Self-attention: captura y mira directamente todas las palabras.

El paper “Attention Is All You Need”, publicado por Ashish Vaswani, Noam Shazeer, Niki Parmar y otros investigadores de Google, introdujo la arquitectura Transformer.

Los beneficios son:

- Capturar dependencias largas.
- Todas las palabras se analizan al mismo tiempo. (mejor uso de GPU).
- Mejor compresión del texto.
- Escala mejor con modelos con más parámetros.

¿Qué son los *tokens*?



Los computadores no procesan palabras, sino que procesan números. (representaciones binarias).

Los tokens es el conjunto de “palabras” que hay en el conjunto de los datos de entrenamiento del modelo.

Ejemplo: Hola, como estas?

Lista de Tokens: [“Hola”, “,como”, “estas?”]

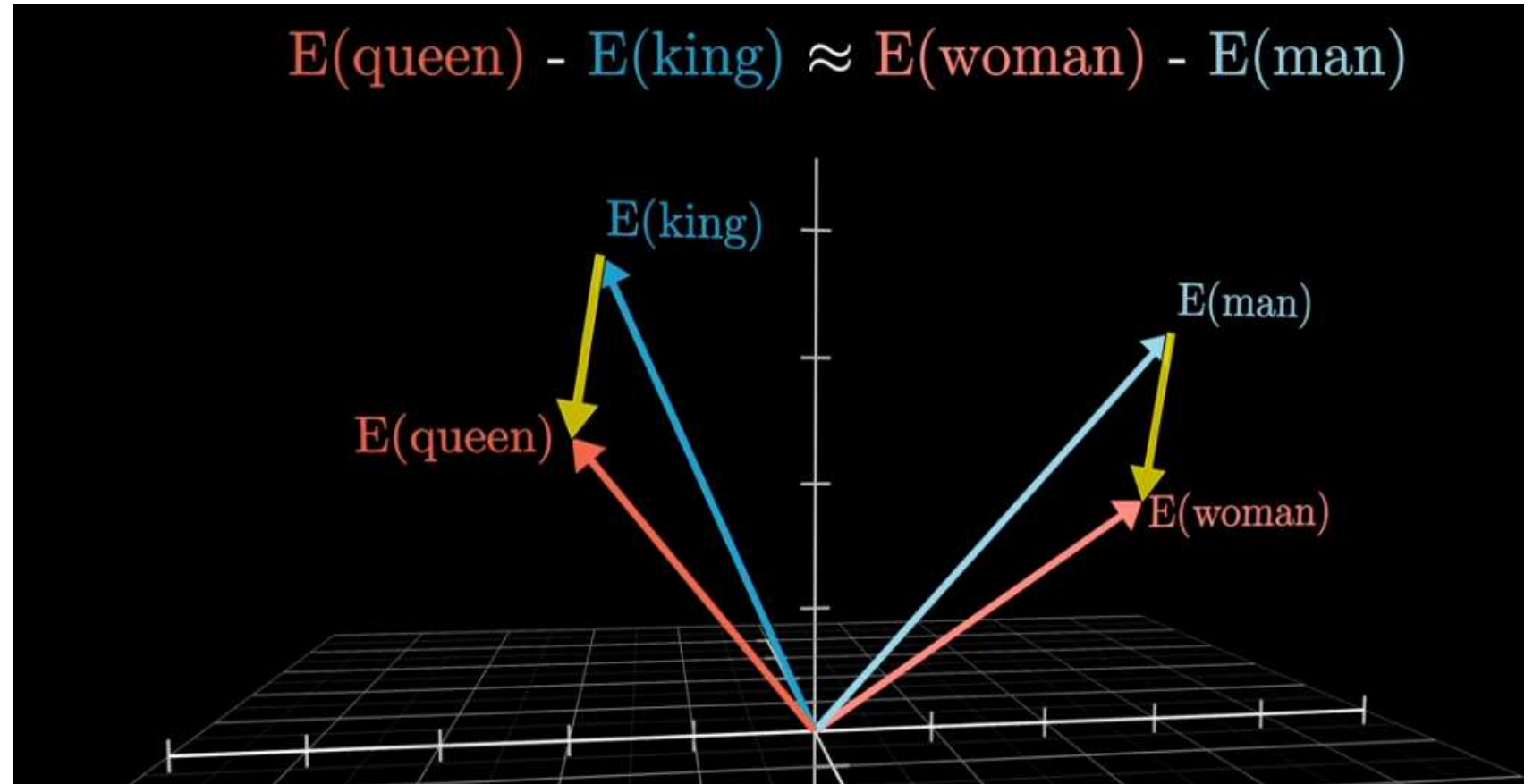
El input del usuario se divide en tokens mediante un tokenizador. Luego se buscan los IDs de esos tokens y cada ID se transforma en un embedding.

Los tokens forman parte central del modelo de negocio de los LLMs, ya que cada consulta realizada a estos sistemas se cobra considerando tanto los tokens de entrada (prompt tokens) como los tokens de salida (completion tokens).

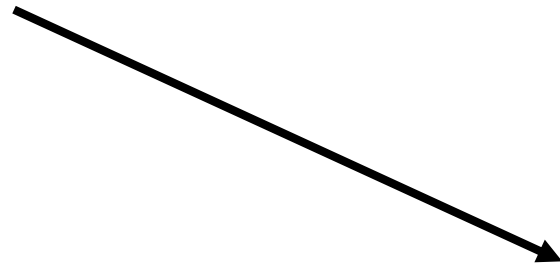
¿Qué son los *Embeddings*?

Los embeddings son vectores de números que representan el significado de palabras, frases o documentos para que un computador pueda procesarlos.

Los embeddings son importantes por el propio modelo, y por otras técnicas, tales como: **RAG**. (**Retrieval-Augmented Generation**).



Prompt Engineering es una técnica para mejorar las respuestas de los modelos, buscando consistencia, reducción de alucinaciones y personalización de los LLMs.



¿Es la solución perfecta?

No, pero ayuda.

¿Cómo construir un buen prompt?

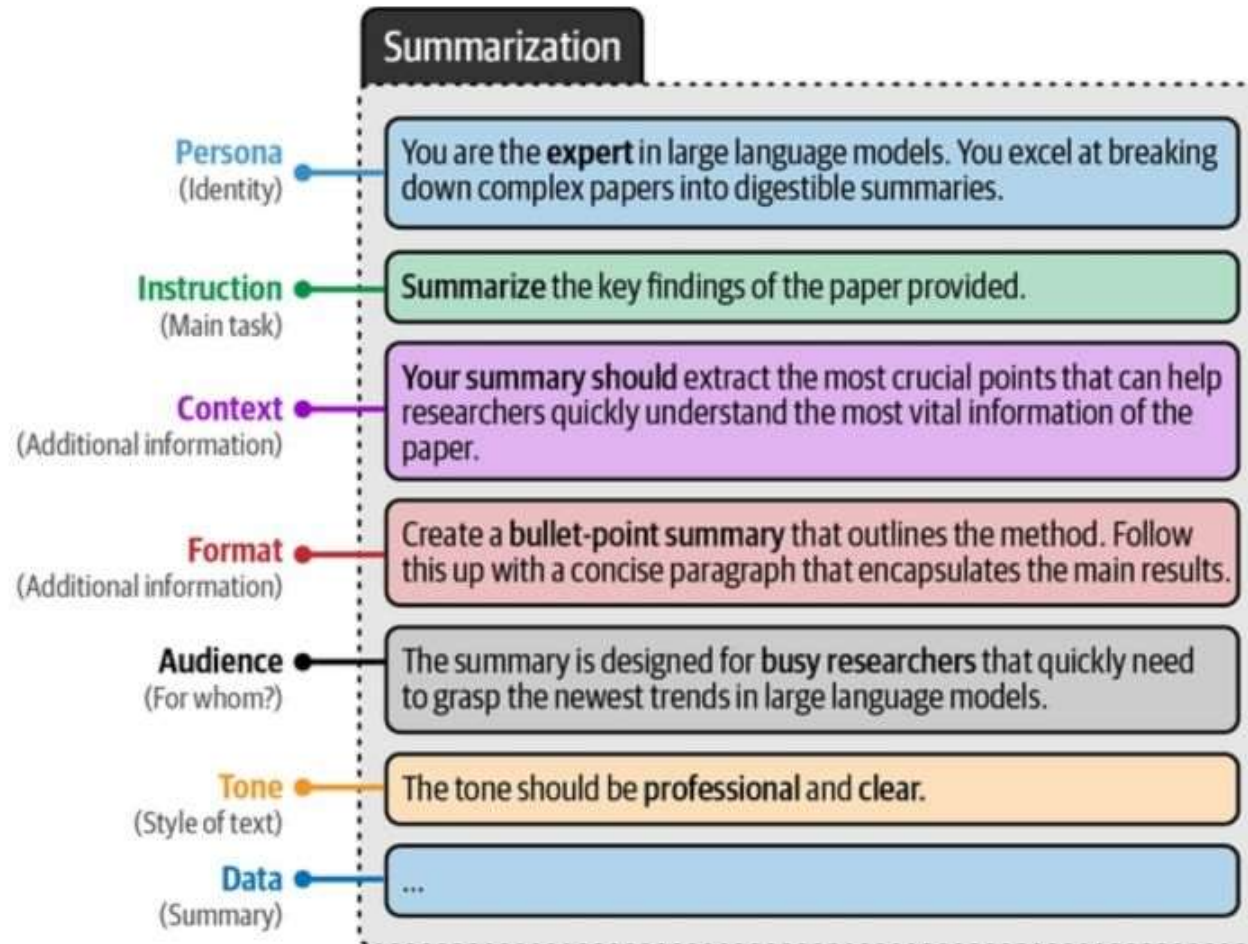
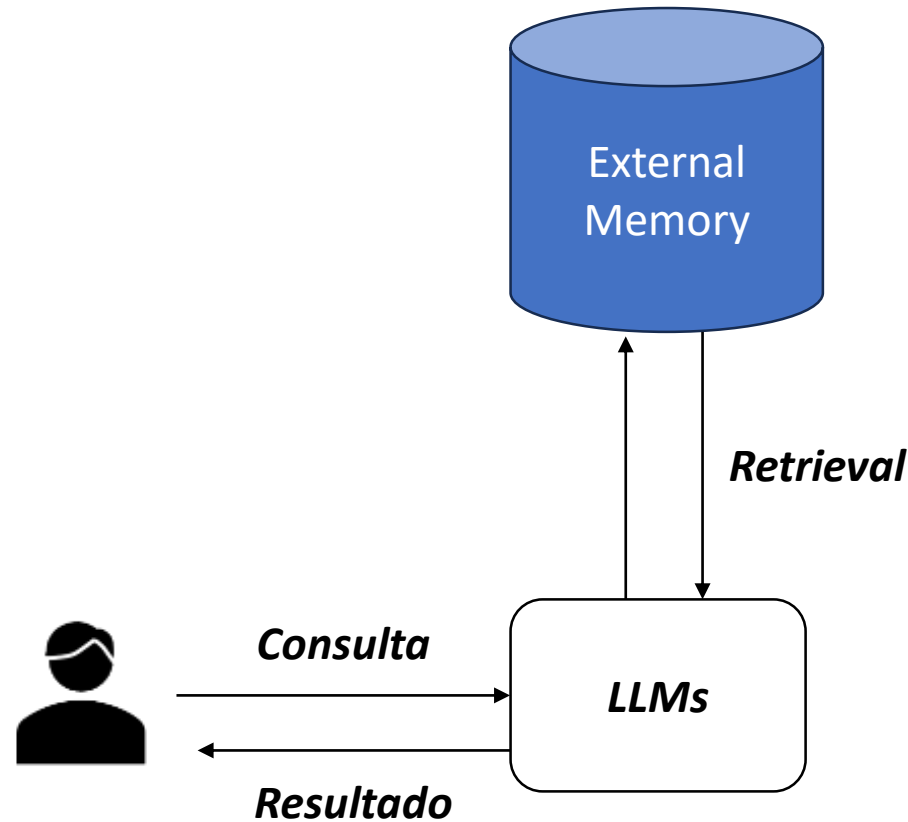


Figure 6-11. An example of a complex prompt with many components.

Simplicity is the ultimate sophistication. (Leonardo da Vinci)

¿Qué es RAG?: Retrieval-Augmented Generation



RAG es una metodología para dotar a los LLMs con información que no tenían al momento de entrenarlos.

1. **Retrieval (Recuperar):** Encontrar las fuentes relevantes a la pregunta.
2. **Aumentar (Augment):** La información recuperada se inyecta al prompt.
3. **Generar (Generative):** El LLM produce una respuesta en base a la información dada.

Retrieval: Dos caminos, mismo objetivo.

Term base retrieval

Term Frequency (TF): Numero de veces que un termino aparece en un documento. Si el termino aparece más veces, es más relevante.

Inverse document frequency (IDF) : La importancia de un termino es inversamente proporcional a la cantidad de documentos donde aparece el termino.



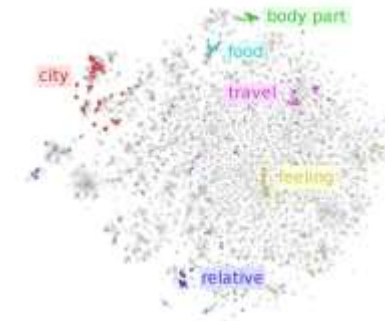
*Las soluciones más típicas a este tipo de retrieval son: **ElasticSearch** y **BM25**.*

Esta solución se enfoca en el lexical level

Embedding base retrieval

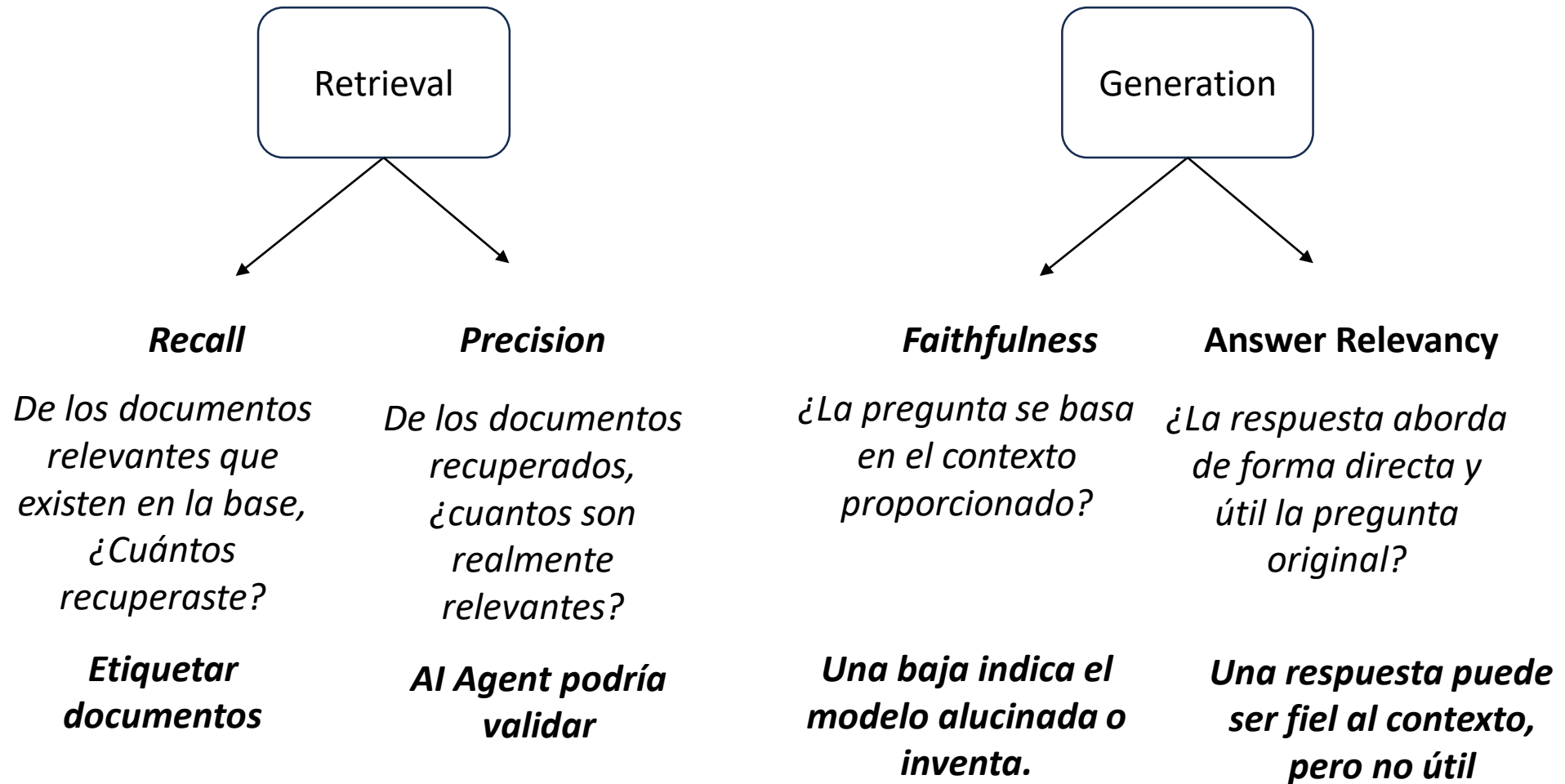
Transforma la información en vectores, se encuentran los más parecidos, y luego se inyectan al modelo de lenguaje como parte del prompt.

Los vectores son capaces de capturar la representación semántica del texto.



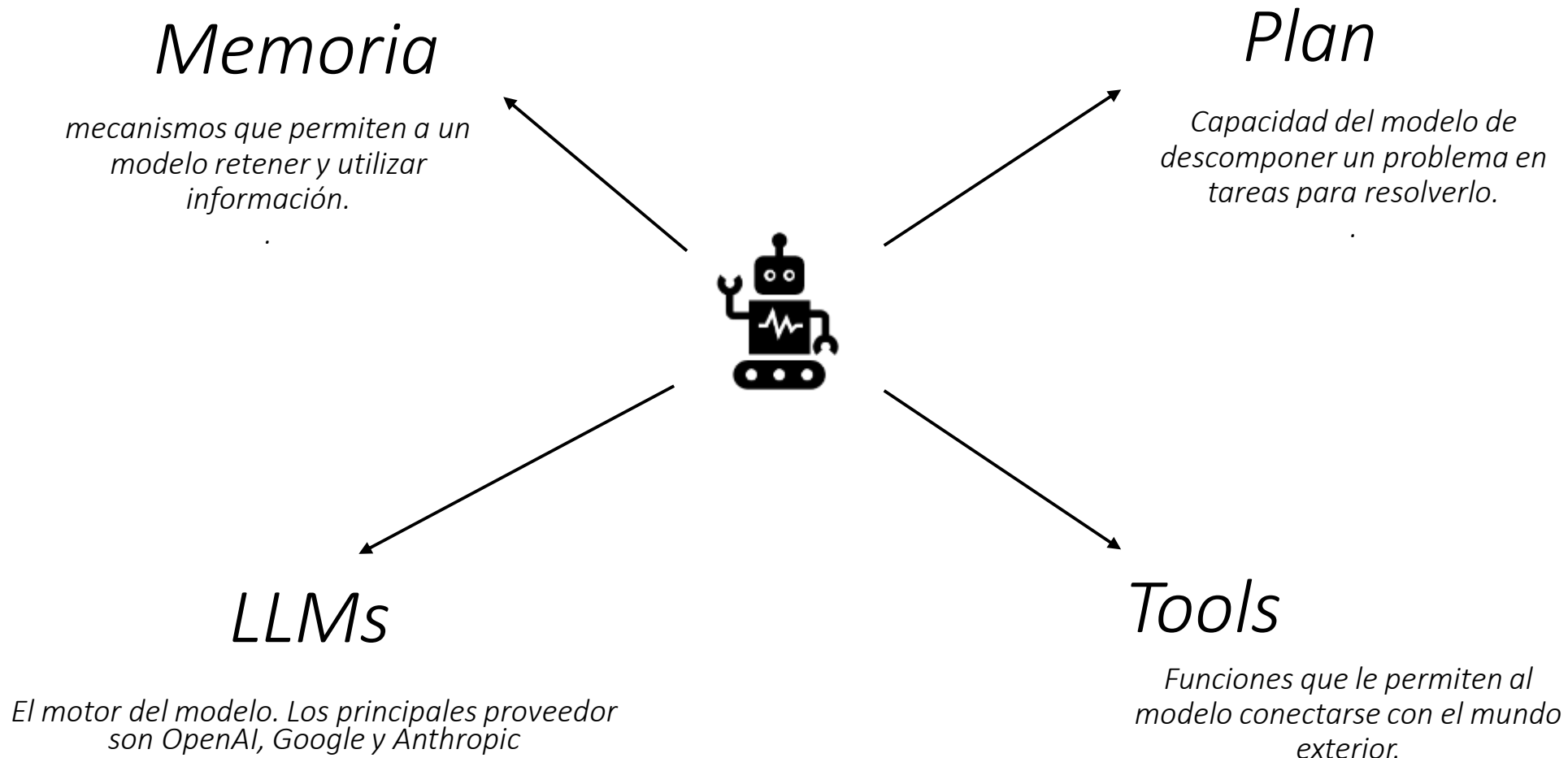
Esta solución se enfoca en el semantic level.

Control: Métricas de Recuperación y Generación.

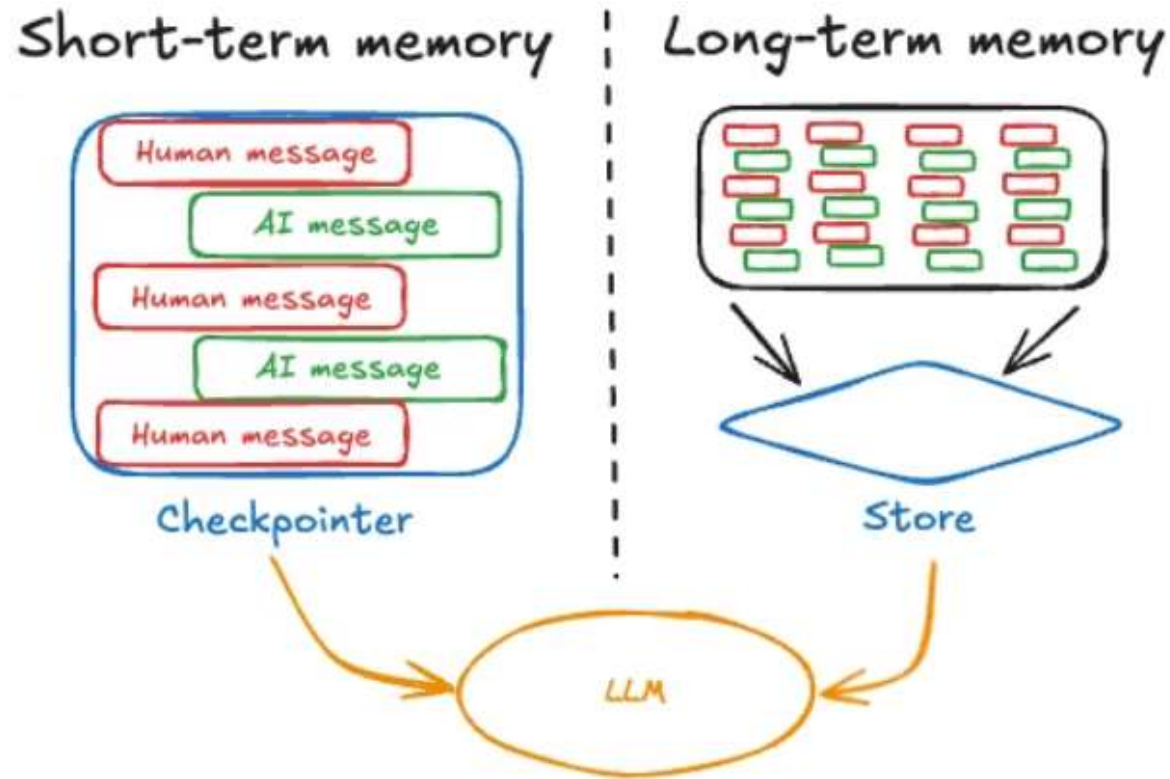


Experiencia de Aprendizaje 2
Desarrollo de Agente
Inteligentes con AI

¿Cuáles son sus componentes?



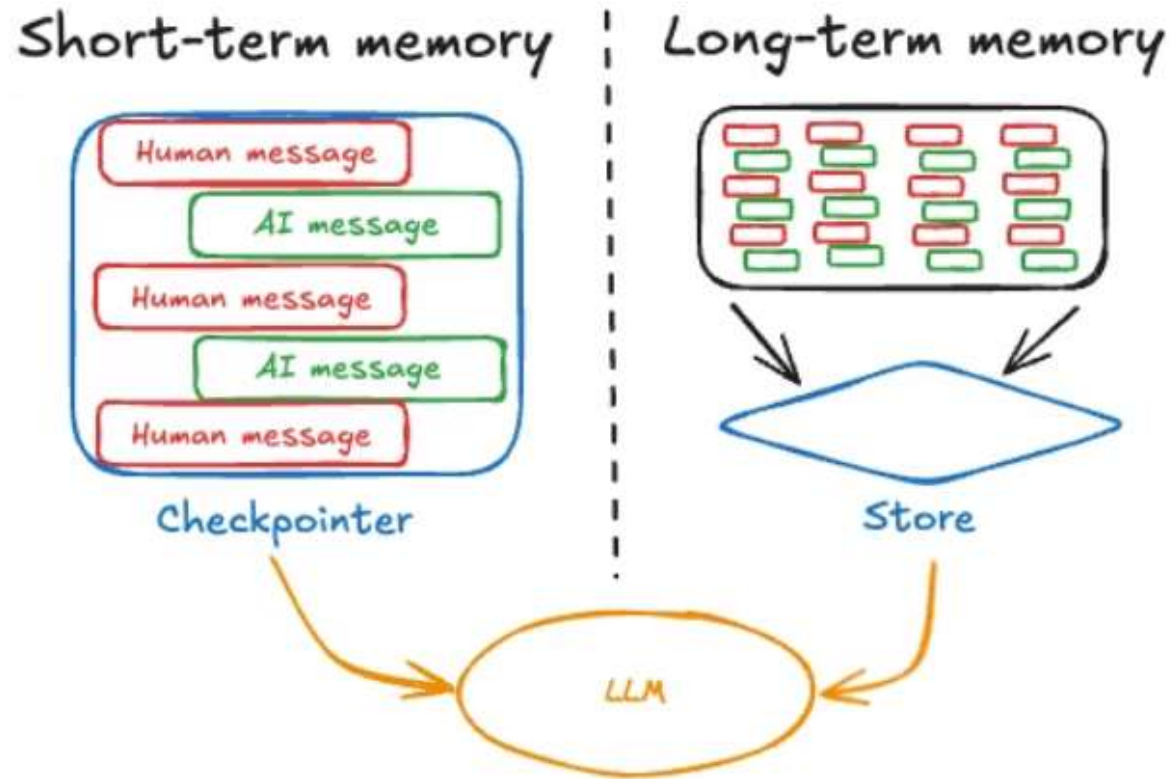
Memoria : Tipo de memoria en los agentes.



Short-term memory: Es todo lo que el agente puede "ver" y recordar en este momento — su ventana de atención activa.

Long-term memory: Es todo lo que el agente puede recordar más allá de su context window — información persistente que sobrevive entre sesiones.

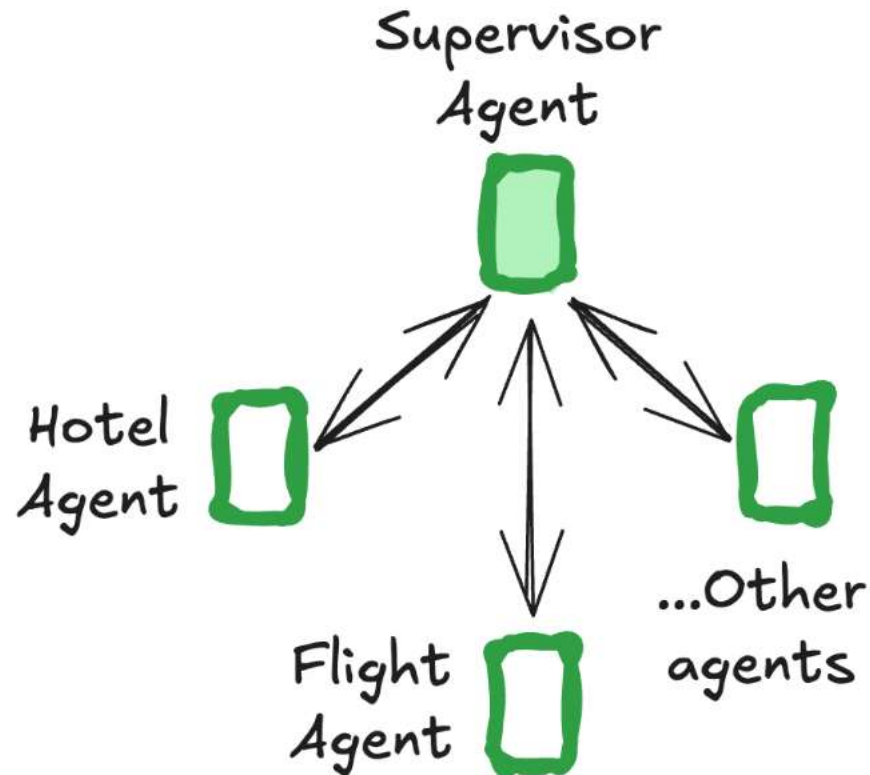
Memoria : Tipo de memoria en los agentes.



Short-term memory: Es todo lo que el agente puede "ver" y recordar en este momento — su ventana de atención activa.

Long-term memory: Es todo lo que el agente puede recordar más allá de su context window — información persistente que sobrevive entre sesiones.

Multi – Agents system: Cuando lo simple no es suficiente.



“In 2025, the models out there are extremely intelligent. But even the smartest human won’t be able to do their job effectively without the context of what they’re being asked to do. “Prompt engineering” was coined as a term for the effort needing to write your task in the ideal format for a LLM chatbot. “Context engineering” is the next level of this. It is about doing this automatically in a dynamic system. It takes more nuance and is effectively the #1 job of engineers building AI agents.”

Multi – Agents system: 4 patrones para mirar.

Pattern	How it works
<u>Subagents</u>	A main agent coordinates subagents as tools. All routing passes through the main agent, which decides when and how to invoke each subagent.
<u>Handoffs</u>	Behavior changes dynamically based on state. Tool calls update a state variable that triggers routing or configuration changes, switching agents or adjusting the current agent's tools and prompt.
<u>Skills</u>	Specialized prompts and knowledge loaded on-demand. A single agent stays in control while loading context from skills as needed.
<u>Router</u>	A routing step classifies input and directs it to one or more specialized agents. Results are synthesized into a combined response.
<u>Custom workflow</u>	Build bespoke execution flows with <u>LangGraph</u> , mixing deterministic logic and agentic behavior. Embed other patterns as nodes in your workflow.

SubAgents: Orquestación centralizada.

¿Cómo funcionan?:

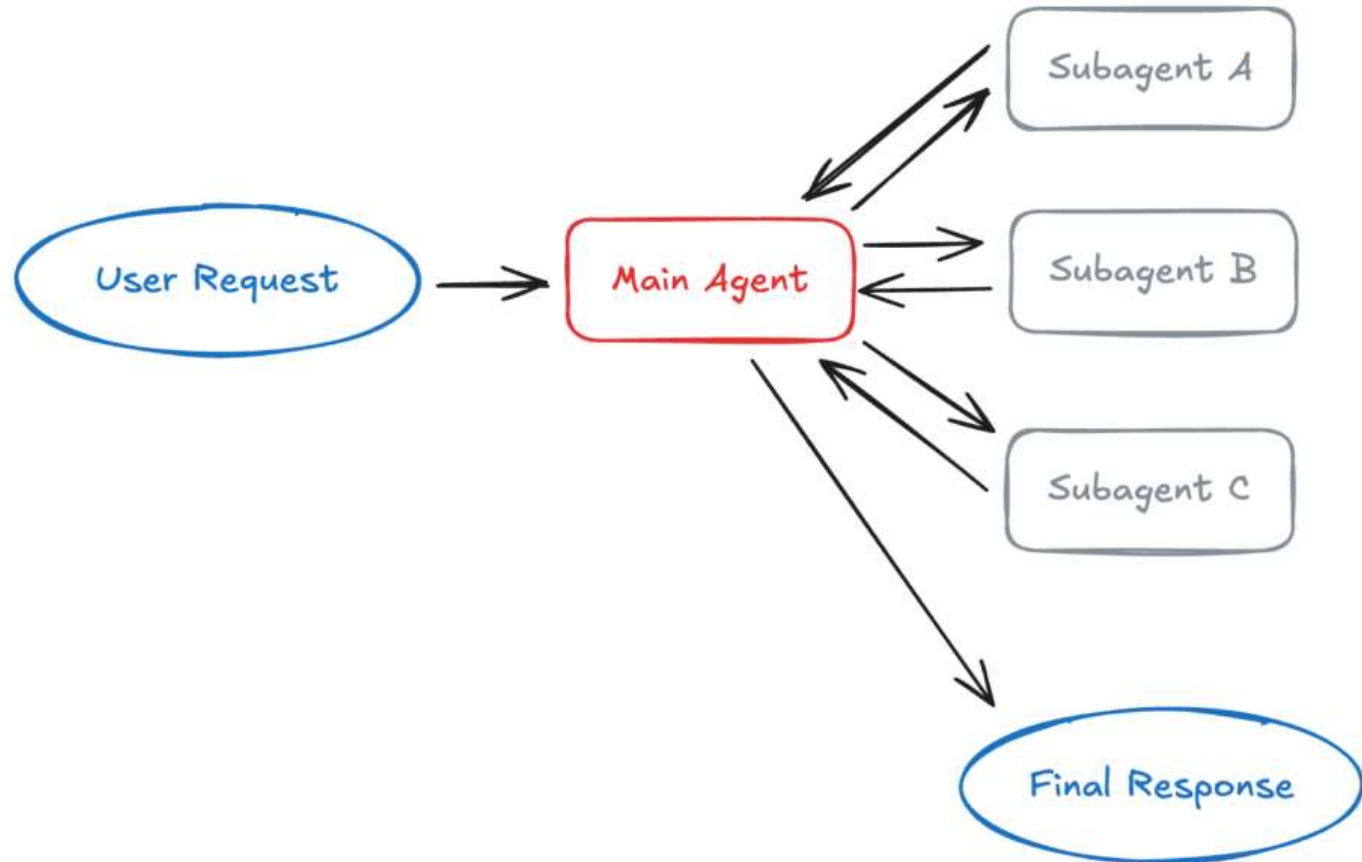
El agente principal decide que subagente invocar, que información proporcionar y como combinar los resultados. El agente principal puede invocar a múltiples agentes en paralelo.

Ideal:

Aplicaciones con múltiples dominios distintos donde se necesita un control centralizado del flujo de trabajo y los subagentes no necesitan conversar directamente con los usuarios.

¿Trade-off?:

Añade una llamada al modelo adicional por interacción, ya que los resultados deben volver a pasar por el agente principal.



Skills: Divulgación progresiva.

¿Cómo funcionan?:

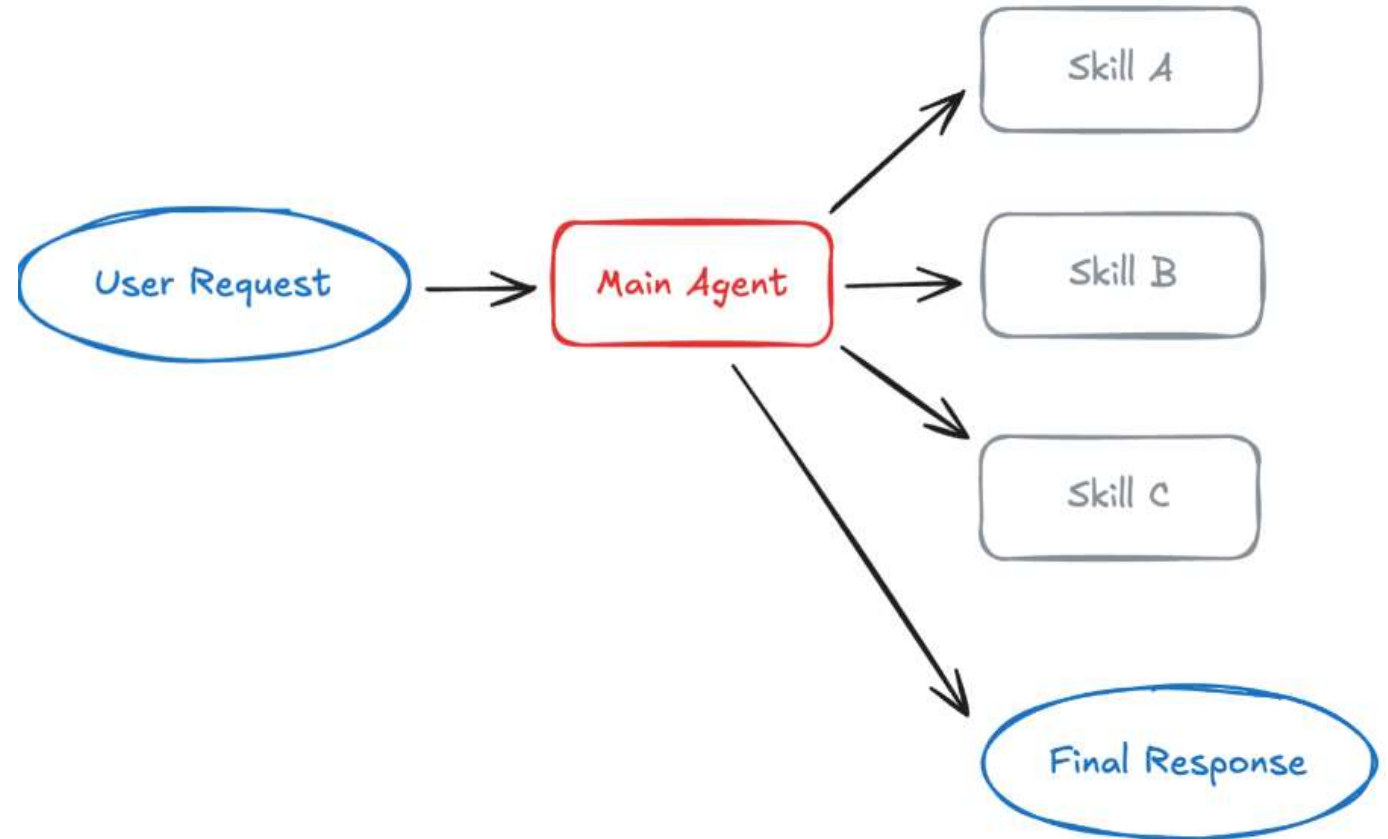
Las habilidades son especializaciones basadas principalmente en prompts, empaquetadas como directorios que contienen instrucciones, scripts y recursos. Al iniciarse, el agente solo conoce los nombres y descripciones de las habilidades. Cuando una habilidad se vuelve relevante, el agente carga su contexto completo.

Ideal:

Agentes únicos con muchas especializaciones posibles, situaciones donde no se necesita imponer restricciones entre capacidades, o equipos distribuidos donde distintos equipos mantienen diferentes habilidades.

¿Trade-off?:

El contexto se acumula en el historial de la conversación a medida que se cargan las habilidades, lo que puede generar una acumulación excesiva de tokens en llamadas posteriores.



Handoffs: Transiciones impulsadas por el estado.

¿Cómo funcionan?:

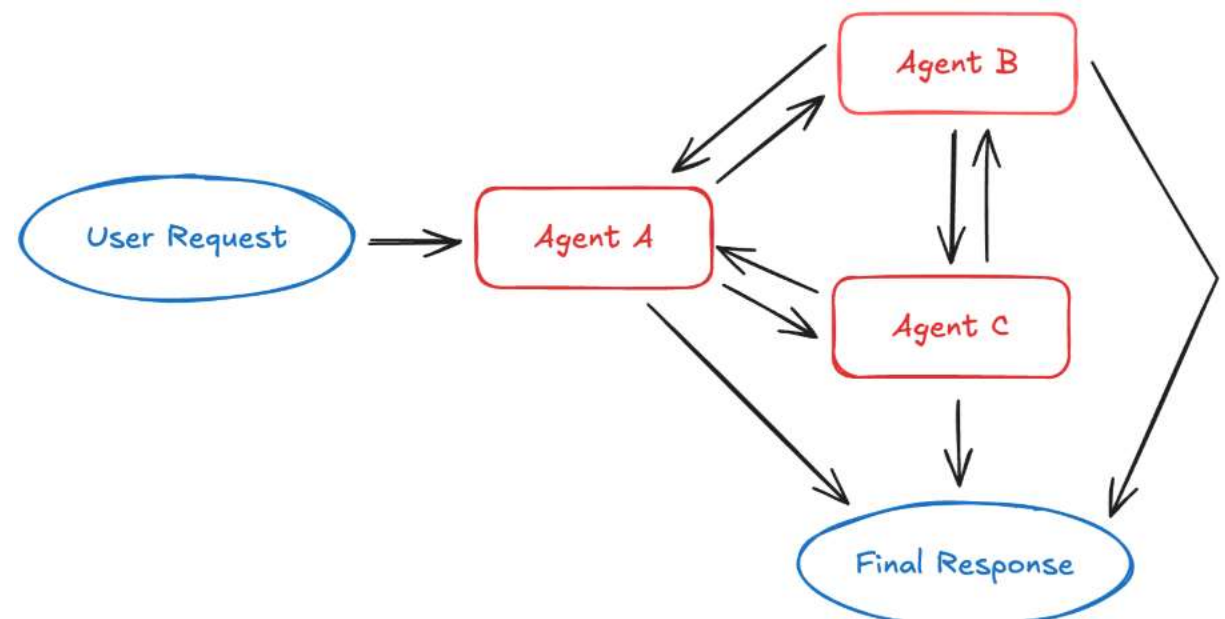
Cuando un agente invoca una herramienta de transferencia, actualiza el estado que determina cuál será el próximo agente en activarse. Esto puede implicar cambiar a un agente diferente o modificar el prompt del sistema y las herramientas disponibles del agente actual.

Ideal:

Flujos de atención al cliente que recopilan información por etapas, experiencias conversacionales en múltiples fases, o cualquier escenario que requiera restricciones secuenciales donde las capacidades se desbloquean solo después de cumplir ciertas condiciones previas.

¿Trade-off?:

Es más dependiente del estado que otros patrones, lo que requiere una gestión cuidadosa del mismo.



Handoffs: Asistente de un banco.

Internamente, el agente interactúa y cambia a otros agentes sin que el usuario expresamente haya guiado a esos nodos.

Situación inicial: El usuario escribe "Quiero hacer una transferencia"

El agente activo es el **Agente de Bienvenida**. Su único trabajo es identificar qué quiere el usuario. Cuando detecta que quiere hacer una transferencia, llama a una herramienta llamada `transferir_a_verificacion()`.

¿Qué pasa internamente cuando se llama esa herramienta?

Se actualiza una variable de estado:

```
estado_actual = "verificacion_identidad"
```

Eso dispara un cambio: ahora el sistema activa el **Agente de Verificación**, que tiene un prompt diferente, y solo tiene acceso a herramientas como `verificar_rut()` o `verificar_clave()`. No puede hacer transferencias todavía.

El usuario pasa la verificación. El agente llama a `transferir_a_operaciones()` y el estado cambia:

```
estado_actual = "operaciones_bancarias"
```

Ahora se activa el **Agente de Operaciones**, que sí tiene acceso a `ejecutar_transferencia()`. Esta herramienta no estaba disponible antes porque no se habían cumplido las condiciones previas.

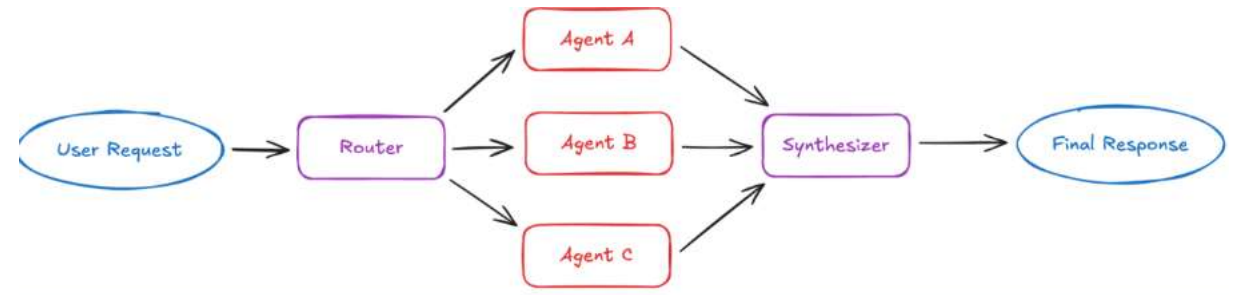
Router: Dirige y ejecuta en paralelo.

¿Cómo funcionan?:

El enrutador descompone la consulta, invoca cero o más agentes especializados en paralelo y sintetiza los resultados en una respuesta coherente.

Ideal:

Aplicaciones con verticales distintas (dominios de conocimiento separados), escenarios que requieren consultas a múltiples fuentes en paralelo, o situaciones donde se necesita sintetizar resultados de múltiples agentes.



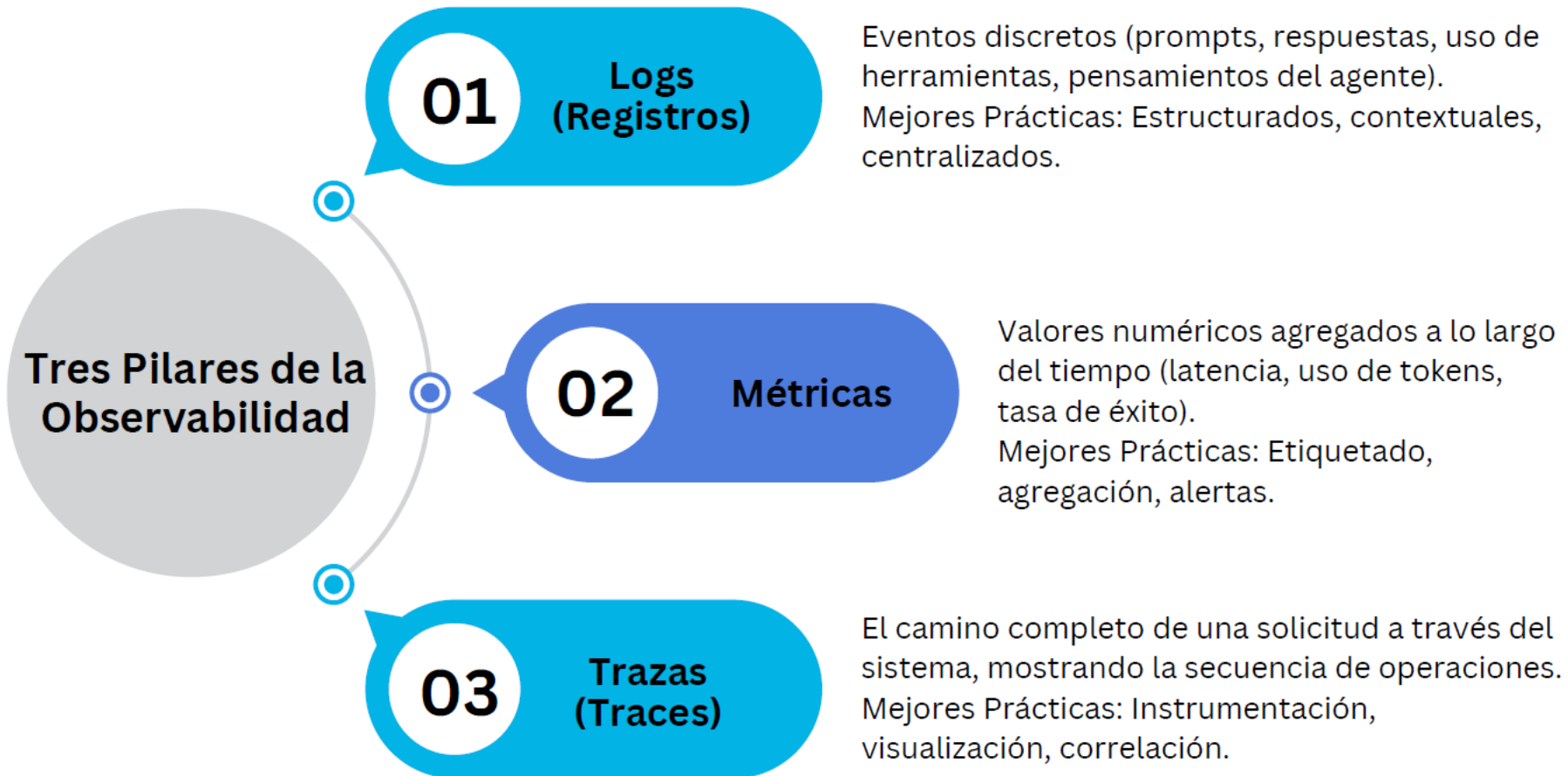
¿Trade-off?:

El diseño sin estado garantiza un rendimiento consistente por solicitud, pero genera una sobrecarga repetida de enrutamiento si se necesita historial de conversación.

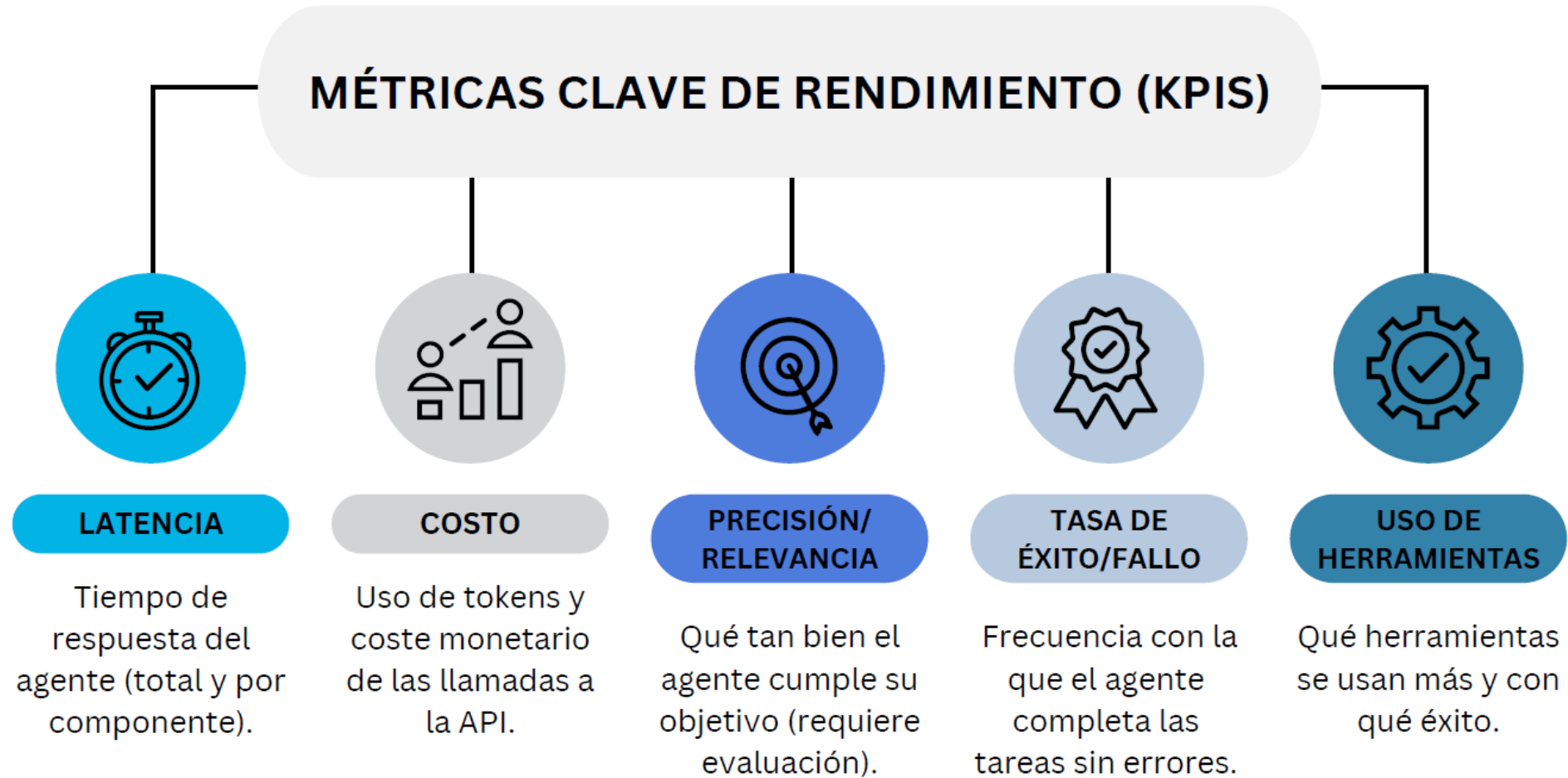
Experiencia de Aprendizaje 3
Observabilidad, Seguridad y ética
de Agente de AI

Experiencia de Aprendizaje 3
Observabilidad, Seguridad y ética
de Agente de AI

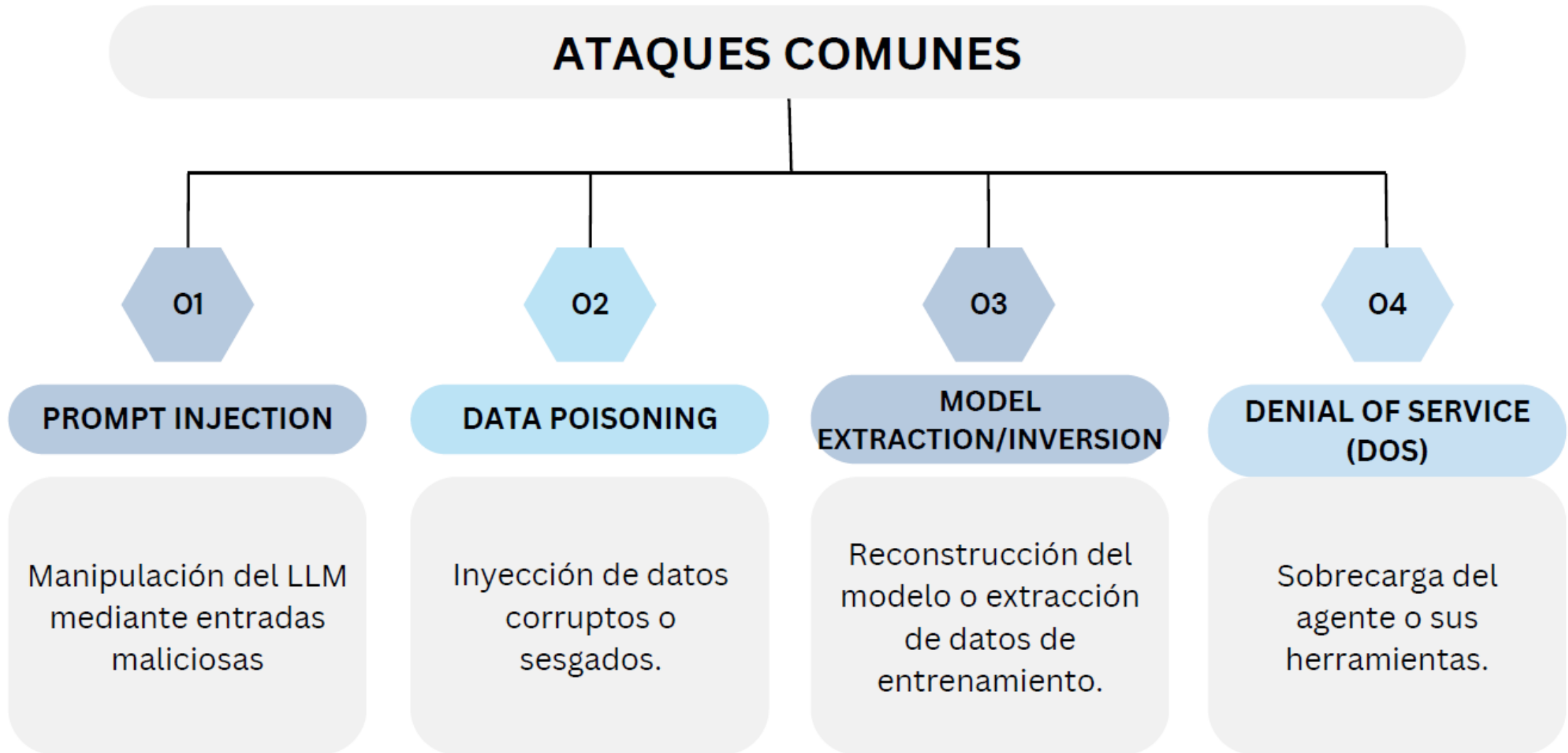
Tres “Stones”: Logs, Métricas e Traces(Trazas).



Métricas: Datos cuantificables del modelo.



Ataques comunes de AI.



Guardrails: Protege tu sistema ante filtraciones.

Inputs Guardrails

Las dos principales fugas que evitan son:

1. filtrar información privada a API externas.
2. ejecutar mensajes maliciosos que comprometen su sistema

¿Cómo puede pasar?:

1. Un empleado copia información confidencial de la empresa o información privada de un usuario en un mensaje y la envía a una API de terceros.¹
2. Un desarrollador de aplicaciones introduce políticas y datos internos de la empresa en el mensaje del sistema de la aplicación.
3. Una herramienta recupera información privada de una base de datos interna y la agrega al contexto.

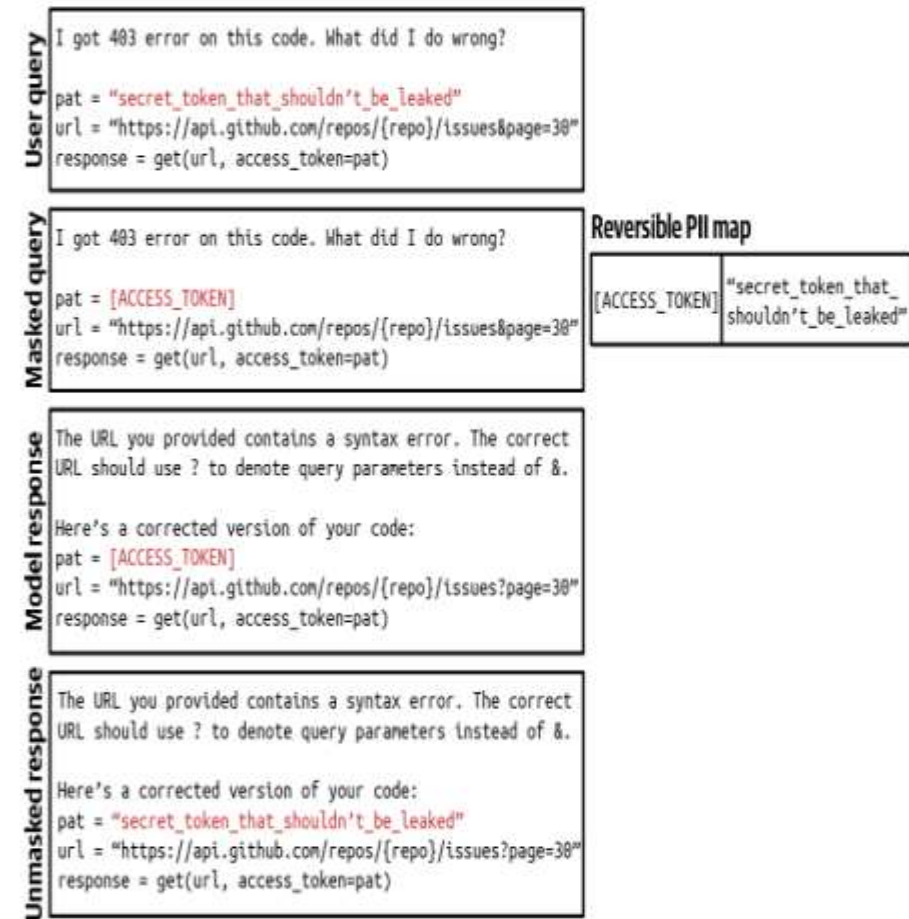


Figure 10-3. An example of masking and unmasking PII information using a reverse PII map to avoid sending it to external APIs.

Guardrails: Protege tu sistema ante filtraciones.

Output Guardrails

Sus funciones son:

1. Detectar fallos de salida.
2. Especificar la política para gestionar los diferentes modos de fallo.

- Quality
 - Malformatted responses that don't follow the expected output format. For example, the application expects JSON, and the model generates invalid JSON.
 - Factually inconsistent responses hallucinated by the model.
 - Generally bad responses. For example, you ask the model to write an essay, and that essay is just bad.
- Security
 - Toxic responses that contain racist content, sexual content, or illegal activities.
 - Responses that contain private and sensitive information.
 - Responses that trigger remote tool and code execution.
 - Brand-risk responses that mischaracterize your company or your competitors.

¿Cómo implementarlo?

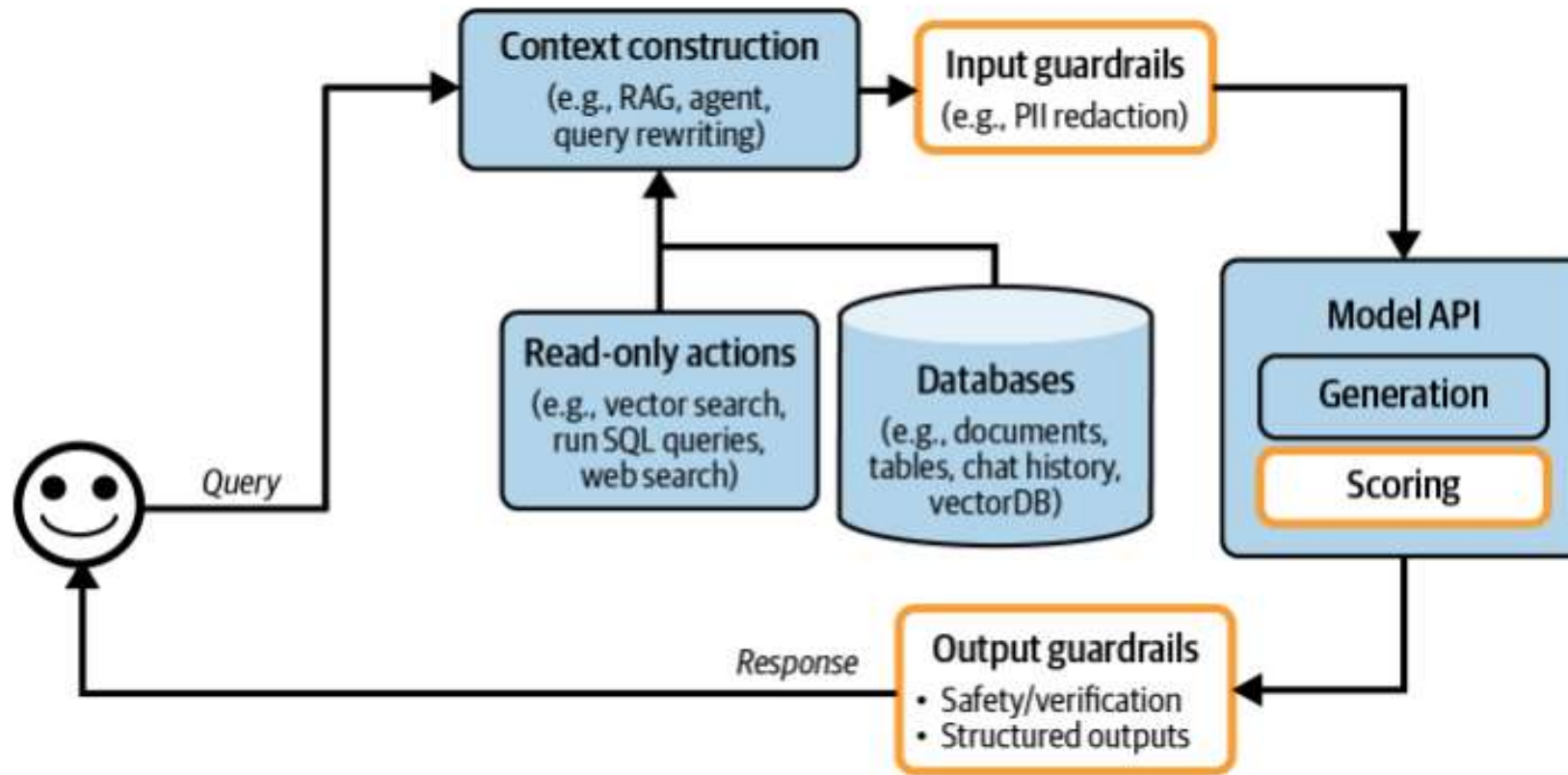


Figure 10-4. Application architecture with the addition of input and output guardrails.

Sistema protegido, con **input y output guardrails** para resolver cualquier problema de filtraciones y seguridad.

¿Cómo implementarlo?

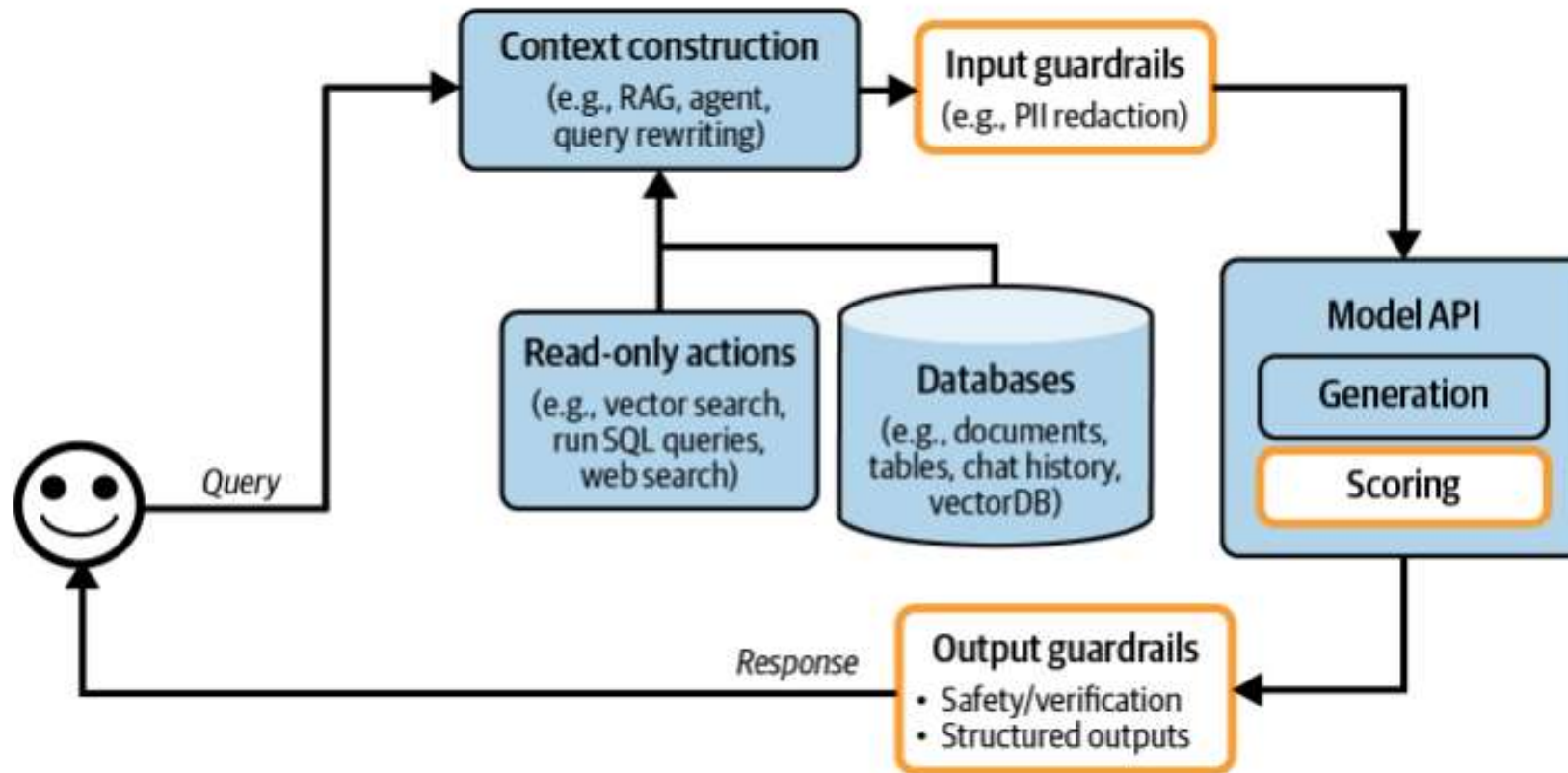
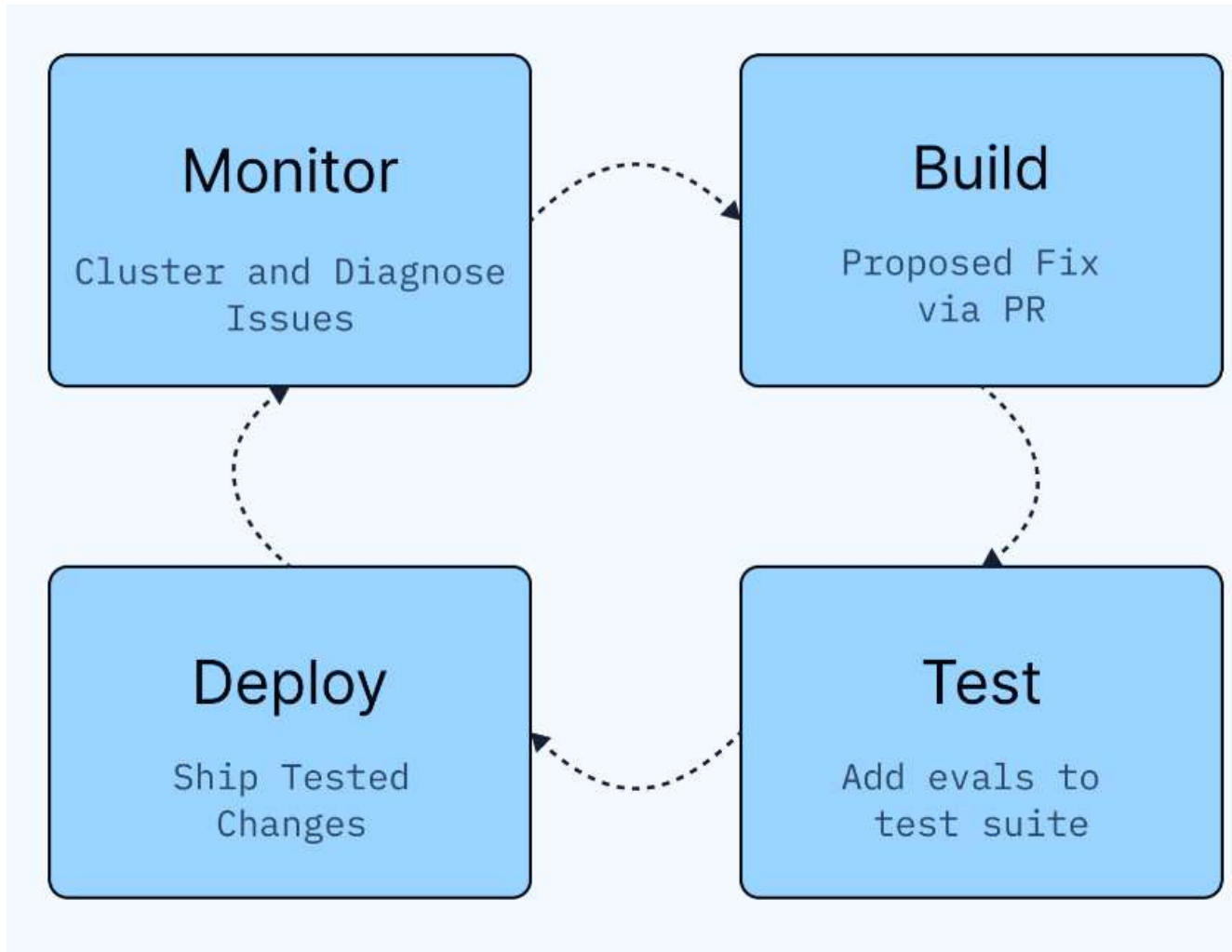


Figure 10-4. Application architecture with the addition of input and output guardrails.

Sistema protegido, con **input y output guardrails** para resolver cualquier problema de filtraciones y seguridad.

LangSmith Engine: El ciclo se cierra.

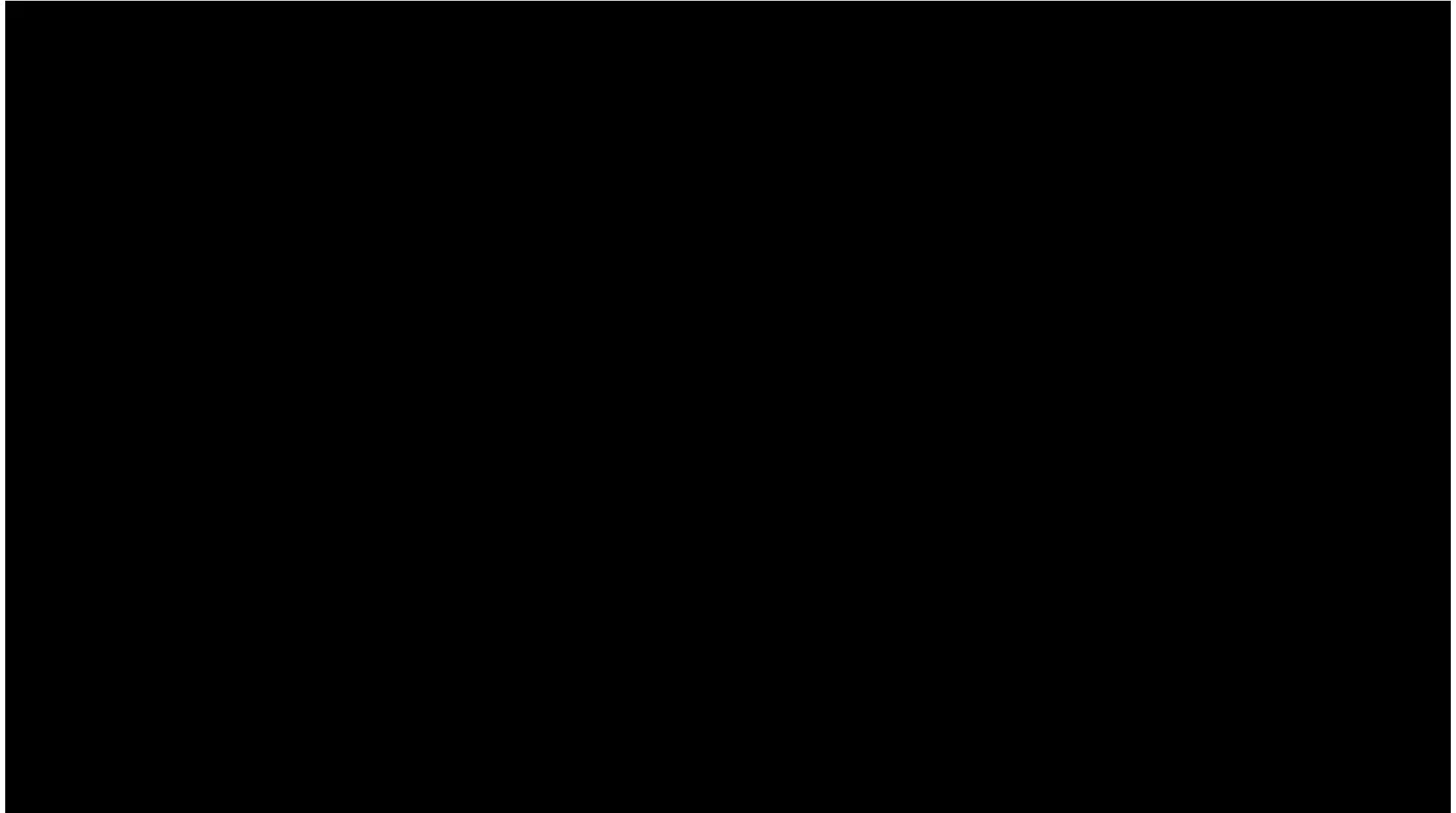


*Consejos, ideas y
pensamientos*

Que me hubiese gustado saber antes...

1. *El cambio es lo único constante.*
2. *Todo lo que aprendan en esta clase posiblemente será obsoleto en 1 o 2 años.*
3. *Lean lo más posible. (Papers, articulo y escuchen a personas).*
4. *Tengan coraje.*
5. *La barrera de construcción software bajaran constamente. Cosas criticas: Infractuctura, Arquitectura (Design patron), Testings y Cyber Seguridad.*
6. *Aprendan cosas diversas. La creatividad viene de juntar distintos puntos de vista.*
7. *Prioriza de rodearme con personas más inteligentes que tu.*

Jim Rohn...*Padre del Desarrollo Personal.*



Muchas gracias

